# Python OOP AIML Guide

**chatgpt.com**/c/6ae088ac-f0a4-83e8-a6fd-7adee78d6f7e

1. oops basics to advanced concept code and 
2. def and interview code and questions
3. very easy word and explanation 
4. AIML real usecase and study also
5. all chapter wise types and its concept

# Python OOP + Functions (def) — Complete AIML Engineer Study Guide

This roadmap will help you learn **Object-Oriented Programming (OOP) and Python functions from beginner to advanced level**, using very easy English, practical code, real-world AI/ML use cases, dry runs, and interview preparation.

The goal is not just to memorize definitions. You should be able to **understand the concept, write code yourself, explain it in an interview, and use it in an AI/ML project.**

# Part 1: Complete chapter-wise roadmap

### Chapter A — Python Functions (def)

Learn how to create reusable blocks of code.

- A1. Function basics, def, calling a function
- A2. Parameters vs arguments
- A3. return vs print
- A4. Positional, keyword and default arguments
- A5. *args and **kwargs
- A6. Local, global and enclosing scope
- A7. Lambda, higher-order functions, map(), filter()
- A8. Recursion, closures and decorators
- A9. Type hints, docstrings and best practices
- A10. Interview coding problems and dry runs

---

### Chapter B — Object-Oriented Programming (OOP)

Learn how to organize code into objects, like real-world things.

- B1. Class and object
- B2. __init__() and self

- B3. Instance vs class variables
- B4. Instance, class and static methods
- B5. Encapsulation and access conventions
- B6. Inheritance and its types
- B7. Polymorphism and method overriding
- B8. Abstraction and abstract classes
- B9. Composition and aggregation
- B10. MRO, super() and multiple inheritance
- B11. Magic methods and properties
- B12. Dataclasses and practical design
- B13. OOP interview questions and coding problems

---

**Chapter C — Apply Functions and OOP to AIML**

- C1. Data preprocessing functions
- C2. Classes for ML datasets and models
- C3. Training and prediction methods
- C4. Evaluation metrics and reusable functions
- C5. Model pipelines and configuration
- C6. Mini-project: reusable ML prediction system
- C7. AIML interview questions and project explanations

---

# Part 2: Python functions (def) — basics to advanced

## Chapter 1 — What is a function?

A **function** is a reusable block of code that performs a particular task.

Real-world example: Imagine a washing machine. You put clothes inside, press a button, and it performs a washing task. In Python, a function works similarly: you provide inputs, it performs a task, and it may return an output.

### Basic syntax

```
def function_name(parameters):
    # Code to perform a task
    return result
```

| Keyword or term | Easy meaning |
|---|---|
| def | Tells Python to create a function |

| Keyword or term | Easy meaning |
|---|---|
| function_name | The name you choose |
| parameters | Input variables defined by the function |
| Function body | Instructions that run when called |
| return | Sends a result back to the caller |

### Example 1: Create and call a function

```
def greet():
    print("Welcome to AI/ML!")

greet()
greet()
```

Output:

```
Welcome to AI/ML!
Welcome to AI/ML!
```

**Dry run:**

1. Python reads def greet() and defines the function.
2. The first greet() call executes its body.
3. The second call executes the same body again.

Important: Defining a function does not execute its body. Calling it does.

## Chapter 2 — Parameters and arguments

- **Parameter:** A variable written in the function definition.
- **Argument:** The actual value supplied when calling the function.

```
def predict_price(area, rate):
    price = area * rate
    return price

result = predict_price(1000, 2500)
print(result)
```

Output:

```
2500000
```

Here, area and rate are parameters. 1000 and 2500 are arguments.

Dry run:

| Step | Operation | Value |
|---|---|---|
| 1 | area = 1000 | 1000 |
| 2 | rate = 2500 | 2500 |
| 3 | area * rate | 2,500,000 |
| 4 | Return the result | 2,500,000 |

AIML connection: This pattern is useful for calculating a predicted price from house features, although a real ML model learns the relationship from training data.

## Chapter 3 — print() vs return

This is one of the most important Python interview topics.

```
def add_print(a, b):
    print(a + b)

def add_return(a, b):
    return a + b

x = add_print(10, 20)
y = add_return(10, 20)

print("x =", x)
print("y =", y)
```

Output:

```
30
30
x = None
y = 30
```

Why?

- print() displays something on the screen.
- return gives the result back to the code that called the function.
- A function without an explicit return value returns None.

For AI/ML, return is especially important because you need to pass calculated values into other functions, metrics, and models.

## Chapter 4 — Types of function arguments

### 4.1 Positional arguments

Values are matched according to their position.

```
def calculate_accuracy(correct, total):
    return correct / total

print(calculate_accuracy(90, 100))
```

Output: 0.9

### 4.2 Keyword arguments

Values are matched by parameter name.

```
def student(name, score):
    print(name, score)

student(score=92, name="Rahul")
```

Output: Rahul 92

### 4.3 Default arguments

A parameter gets a default value if you don't provide one.

```
def train_model(epochs=10):
    print("Training for", epochs, "epochs")

train_model()
train_model(epochs=50)
```

Output:

```
Training for 10 epochs
Training for 50 epochs
```

### 4.4 Variable-length positional arguments — *args

Use *args when a function should accept any number of positional arguments.

```
def average(*scores):
    if not scores:
        return None
    return sum(scores) / len(scores)

print(average(80, 90, 100))
```

Output: 90.0

Here, scores becomes a tuple: (80, 90, 100).

### 4.5 Variable-length keyword arguments — **kwargs

Use **kwargs to accept multiple named arguments.

```
def show_config(**settings):
    for key, value in settings.items():
        print(key, "=", value)

show_config(model="RandomForest", trees=100, seed=42)
```

Output:

```
model = RandomForest
trees = 100
seed = 42
```

AIML use case: Passing flexible model settings to a training function.

## Chapter 5 — Variable scope

Scope means **where a variable can be accessed**.

```
learning_rate = 0.01  # Global variable

def train():
    epochs = 10       # Local variable
    print(learning_rate)
    print(epochs)

train()
```

Output:

```
0.01
10
```

- Local: Created inside a function.
- Global: Defined outside functions at module level.
- Enclosing: Defined in an outer function containing an inner function.
- Built-in: Names Python provides, such as `len()` and `sum()`.

Python looks up names using the **LEGB rule**: Local → Enclosing → Global → Built-in.

```
def outer():
    model_name = "RandomForest"

    def inner():
        print(model_name)

    inner()

outer()
```

Output: `RandomForest`

For most ML code, pass values as function parameters instead of changing global variables. It makes experiments easier to reproduce and debug.

## Chapter 6 — Lambda, `map()` and `filter()`

### Lambda function

A lambda is a small anonymous function written in one expression.

```
square = lambda number: number ** 2
print(square(5))
```

Output: 25

For complex logic, prefer a normal def function.

### map()

Applies a function to every item in an iterable.

```
features = [10, 20, 30]

scaled = list(map(lambda x: x / 10, features))
print(scaled)
```

Output: [1.0, 2.0, 3.0]

### filter()

Keeps items that satisfy a condition.

```
scores = [45, 80, 92, 35, 70]

passed = list(filter(lambda score: score >= 50, scores))
print(passed)
```

Output: [80, 92, 70]

AIML use case: Filtering records that satisfy a quality rule before analysis. In real data preprocessing, also check missing values and whether removing records could bias your dataset.

## Chapter 7 — Recursion

Recursion means a function calls itself.

```
def factorial(n):
    if n < 0:
        raise ValueError("n must be non-negative")
    if n == 0:
        return 1
    return n * factorial(n - 1)

print(factorial(4))
```

Output: 24

Dry run:

| Call | Calculation | Result |
|---|---|---|
| factorial(0) | Base case | 1 |
| factorial(1) | 1 * 1 | 1 |
| factorial(2) | 2 * 1 | 2 |
| factorial(3) | 3 * 2 | 6 |
| factorial(4) | 4 * 6 | 24 |

The base case stops recursion. Without it, the function may call itself until Python raises a recursion-depth error.

## Chapter 8 — Advanced functions

| Concept | Easy meaning | AIML application |
|---|---|---|
| Closure | An inner function remembers values from an outer function | Reusable configuration |
| Decorator | Wraps a function to add behavior | Logging training time |
| Generator (yield) | Produces values one at a time | Processing large datasets |
| Type hints | Describe expected input/output types | More readable ML utilities |
| Docstring | Explains what a function does | Documenting preprocessing steps |
| Higher-order function | Accepts or returns another function | Reusable transformations |

Example of a type-hinted function with a docstring:

```
def normalize(value: float, minimum: float, maximum: float) -> float:
    """Scale a value to the range 0 to 1."""
    if maximum <= minimum:
        raise ValueError("maximum must be greater than minimum")
    return (value - minimum) / (maximum - minimum)

print(normalize(75, 0, 100))
```

Output: 0.75

This is a simple min-max scaling calculation. For production ML, use the training-set minimum and maximum and reuse those same values on validation and test data.

---

# Part 3: OOP — Object-Oriented Programming

## Chapter 1 — What is OOP?

OOP is a way of writing programs by organizing related data and behavior into **objects**.

Think about an ML model. It has data, such as learned parameters, and behaviors, such as training and prediction. A Python class can organize those related parts together.

**Class: MLModel**

Blueprint or design

**Attributes**

model_name, accuracy

**Methods**

train(), predict()

**Objects**

model_1, model_2 — separate instances of the class

## Chapter 2 — Class and object

```
class MLModel:
    def predict(self, temperature):
        return temperature * 2

model_1 = MLModel()
model_2 = MLModel()

print(model_1.predict(25))
print(model_2.predict(30))
```

Output:

```
50
60
```

- class MLModel: Defines the blueprint.
- model_1 and model_2: Objects created from the class.
- predict(): A method, meaning a function defined inside a class.
- self: Refers to the particular instance on which the method is called.

This example illustrates class structure, not a trained prediction model.

## Chapter 3 — __init__() and self

__init__() initializes a new object's attributes when you create an instance.

```
class Student:
    def __init__(self, name, score):
        self.name = name
        self.score = score

    def display(self):
        print(self.name, self.score)

student_1 = Student("Aman", 90)
student_2 = Student("Riya", 95)

student_1.display()
student_2.display()
```

Output:

```
Aman 90
Riya 95
```

Dry run:

| Step | Action | Result |
|---|---|---|
| 1 | `Student("Aman", 90)` | Creates first object |
| 2 | `self.name = name` | Stores `"Aman"` |
| 3 | `self.score = score` | Stores `90` |
| 4 | `Student("Riya", 95)` | Creates a separate object |
| 5 | Call `display()` | Prints each object's own data |

Remember: `self` is the conventional instance parameter name. Python supplies the instance automatically when you call an instance method like `student_1.display()`.

## Chapter 4 — Instance variables vs class variables

```
class MLExperiment:
    framework = "scikit-learn"  # Class variable

    def __init__(self, model_name):
        self.model_name = model_name  # Instance variable

exp_1 = MLExperiment("DecisionTree")
exp_2 = MLExperiment("RandomForest")

print(exp_1.framework)
print(exp_2.framework)
print(exp_1.model_name)
print(exp_2.model_name)
```

Output:

```
scikit-learn
scikit-learn
DecisionTree
RandomForest
```

- A class variable is shared at the class level.

- An instance variable belongs to a particular object.

Avoid using a mutable class variable such as `results =` when you want each experiment to have its own list. Put it in `__init__()` instead.

## Chapter 5 — Types of methods

### Instance method

Receives `self`; works with one object's data.

### Class method

Uses `@classmethod` and receives `cls`; works with the class itself.

### Static method

Uses `@staticmethod`; a related utility that doesn't need automatic `self` or `cls`.

```
class Experiment:
    framework = "Python"

    def __init__(self, name):
        self.name = name

    def describe(self):
        return f"{self.name} uses {self.framework}"

    @classmethod
    def change_framework(cls, framework):
        cls.framework = framework

    @staticmethod
    def valid_score(score):
        return 0 <= score <= 1

exp = Experiment("Classifier")

print(exp.describe())
print(Experiment.valid_score(0.95))

Experiment.change_framework("Python ML")
print(exp.describe())
```

Output:

```
Classifier uses Python
True
Classifier uses Python ML
```

Use an instance method for experiment-specific behavior, a class method for class-wide behavior, and a static method for a related helper that doesn't need object or class state.

## Chapter 6 — The four pillars of OOP

These are among the most frequently asked OOP interview concepts.

**1. Encapsulation**

Keep data and the methods that work with it together. Control how data is changed.

Example: An ML experiment validates its score before updating it.

---

**2. Inheritance**

A child class reuses or extends functionality from a parent class.

Example: Different ML model classes inherit a shared interface.

---

**3. Polymorphism**

The same method name or interface can behave differently for different objects.

Example: A classifier and a regressor can both provide a `predict()` method.

---

**4. Abstraction**

Show what a component can do without requiring users to know all its internal details.

Example: Call `model.fit(X, y)` without implementing the training algorithm yourself.

## 6.1 Encapsulation — practical code

Python uses naming conventions and properties to control access. A single leading underscore means internal use by convention; double leading underscores trigger name mangling, not absolute privacy.

```
class Experiment:
    def __init__(self):
        self._accuracy = 0.0

    @property
    def accuracy(self):
        return self._accuracy

    def set_accuracy(self, value):
        if not 0 <= value <= 1:
            raise ValueError("Accuracy must be between 0 and 1")
        self._accuracy = value

exp = Experiment()
exp.set_accuracy(0.92)
print(exp.accuracy)
```

Output: 0.92

The validation helps prevent invalid values from being stored through the public method.

## 6.2 Inheritance — practical code

```
class MLModel:
    def __init__(self, name):
        self.name = name

    def describe(self):
        return f"Model: {self.name}"

class Classifier(MLModel):
    def predict(self, features):
        return "class_A"

model = Classifier("ImageClassifier")

print(model.describe())
print(model.predict([0.2, 0.8]))
```

Output:

```
Model: ImageClassifier
class_A
```

Classifier inherits __init__() and describe() from MLModel. It adds its own predict() method.

**Common types of inheritance:**

- Single: One parent, one child.
- Multilevel: Grandparent → parent → child.
- Multiple: One child inherits from multiple parents.
- Hierarchical: Multiple children inherit from one parent.
- Hybrid: A combination of inheritance patterns.

## 6.3 Polymorphism — practical code

```
class Classifier:
    def predict(self, data):
        return "Predicted category"

class Regressor:
    def predict(self, data):
        return 25.6

models = [Classifier(), Regressor()]

for model in models:
    print(model.predict([1, 2, 3]))
```

Output:

```
Predicted category
25.6
```

The same call, `model.predict(...)`, works with both objects, but the result differs.

Python supports this through duck typing: if an object supports the required operation, code can often use it without requiring a shared parent class.

## 6.4 Abstraction — practical code

Python provides the `abc` module to define abstract base classes.

```
from abc import ABC, abstractmethod

class MLModel(ABC):
    @abstractmethod
    def predict(self, features):
        pass

class SimpleModel(MLModel):
    def predict(self, features):
        return sum(features) / len(features)

model = SimpleModel()
print(model.predict([10, 20, 30]))
```

Output: 20.0

The abstract base class requires concrete subclasses to implement predict(). You cannot instantiate MLModel itself while it has unimplemented abstract methods.

This simple model calculates an average; it demonstrates the interface, not machine learning.

## Chapter 7 — super() and method overriding

**Method overriding** means a child class supplies its own implementation of a method inherited from a parent.

super() lets you call the next implementation in Python's method resolution order (MRO).

```
class MLModel:
    def __init__(self, name):
        self.name = name

    def describe(self):
        return f"Model: {self.name}"

class NeuralNetwork(MLModel):
    def __init__(self, name, layers):
        super().__init__(name)
        self.layers = layers

    def describe(self):
        return f"{super().describe()}, layers={self.layers}"

model = NeuralNetwork("ImageNetModel", 5)
print(model.describe())
```

Output:

```
Model: ImageNetModel, layers=5
```

Why use `super()`? It avoids duplicating the parent's initialization logic and helps support cooperative multiple inheritance.

## Chapter 8 — Composition vs inheritance

Composition means one object **contains or uses another object**. It is often a good choice when components have different responsibilities.

```
class Preprocessor:
    def transform(self, data):
        return [value / 100 for value in data]

class PredictionService:
    def __init__(self, preprocessor):
        self.preprocessor = preprocessor

    def prepare_data(self, data):
        return self.preprocessor.transform(data)

prep = Preprocessor()
service = PredictionService(prep)

print(service.prepare_data([20, 50, 80]))
```

Output:

```
[0.2, 0.5, 0.8]
```

Here, `PredictionService` uses a `Preprocessor`. It doesn't need to inherit from it.

- Prefer **inheritance** when there is a genuine *is-a* relationship.
- Prefer **composition** when one component *uses or contains* another.

## Chapter 9 — Special methods and properties

Python special methods, often called *dunder methods*, use double underscores.

| Method | Purpose |
| --- | --- |
| `__init__()` | Initialize an object |
| `__str__()` | Human-friendly string representation |

| Method | Purpose |
|---|---|
| __repr__() | Developer-oriented representation |
| __len__() | Define behavior for len(object) |
| __eq__() | Define equality comparison |
| __call__() | Make an object callable like a function |

Example:

```
class Dataset:
    def __init__(self, samples):
        self.samples = samples

    def __len__(self):
        return len(self.samples)

    def __str__(self):
        return f"Dataset with {len(self)} samples"

data = Dataset([10, 20, 30, 40])

print(len(data))
print(data)
```

Output:

```
4
Dataset with 4 samples
```

A related advanced concept is a @property: it allows you to expose a method through attribute-style access while retaining validation or computed behavior.

---

# Part 4: AIML real-world mini-project

Let's combine functions and OOP in a small, beginner-friendly ML workflow.

**Project:** A reusable student-score prediction pipeline.

This example uses a simple rule to predict a score from study hours. It does not train a real ML model, but its structure teaches you how preprocessing, prediction, and evaluation can be separated into reusable components.

```
# 1. Reusable function
def prepare_features(study_hours):
    if study_hours < 0:
        raise ValueError("Study hours cannot be negative")
    return study_hours

# 2. Class for prediction
class ScorePredictor:
    def __init__(self, rate=10):
        self.rate = rate

    def predict(self, study_hours):
        hours = prepare_features(study_hours)
        return min(hours * self.rate, 100)

# 3. Evaluation function
def mean_absolute_error(actual, predicted):
    if len(actual) != len(predicted) or not actual:
        raise ValueError("Lists must have equal, nonzero length")

    errors = [
        abs(a - p)
        for a, p in zip(actual, predicted)
    ]
    return sum(errors) / len(errors)

# 4. Use the system
model = ScorePredictor(rate=10)

hours = [2, 5, 8]
predictions = [model.predict(h) for h in hours]

actual_scores = [25, 55, 75]
mae = mean_absolute_error(actual_scores, predictions)

print("Predictions:", predictions)
print("MAE:", mae)
```

Output:

```
Predictions: [20, 50, 80]
MAE: 5.0
```

## Understand the workflow

### Input data

Study hours: 2, 5, 8

**Function:** prepare_features()

Checks that inputs are valid

**Class:** ScorePredictor

Produces predictions

**Function:** mean_absolute_error()

Compares predictions with actual scores

MAE is the mean absolute error:

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n} |y_i - \hat{y}_i|$$

Here, $y_i$ is the actual score, $\hat{y}_i$ is the predicted score, and $n$ is the number of samples. Lower MAE means smaller average absolute prediction errors.

In a real AIML project, you would typically replace the hand-written prediction rule with a trained estimator, such as LinearRegression, and use appropriate training, validation, and test data.

---

# Part 5: Interview questions with easy answers

## A. Python function interview questions

## B. OOP interview questions

# Part 6: Coding practice for an AIML engineer

Solve these in order, starting with functions and then moving to OOP.

## Practice checklist

(/)

[Learn all solutions step by step]

# Part 7: Your 14-day study plan

| Day | Topic | Daily practice |
|---|---|---|
| 1 | def, parameters and arguments | Write 10 small functions |
| 2 | return, defaults, *args, **kwargs | Solve 8 function problems |

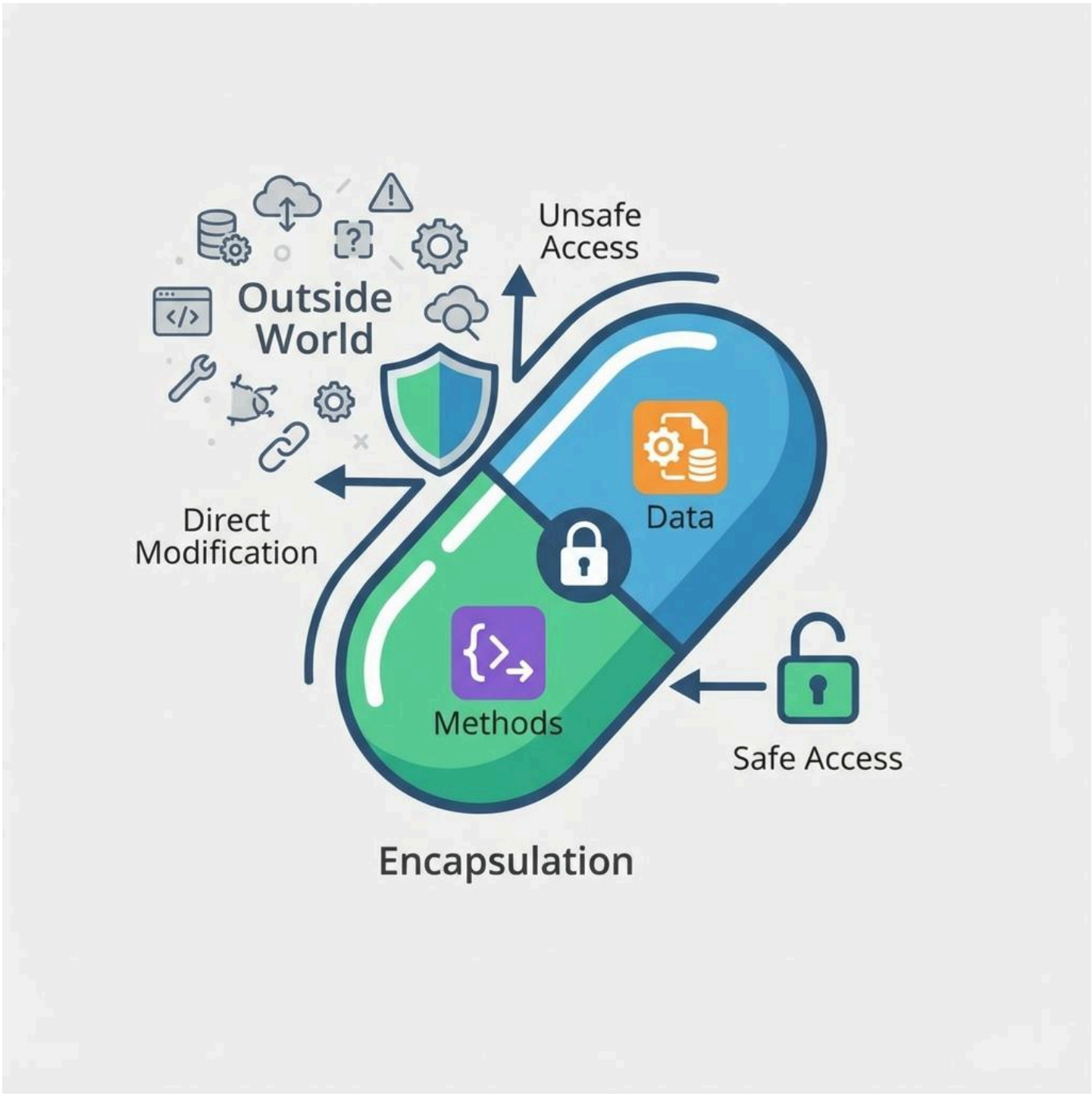
| Day | Topic | Daily practice |
|---|---|---|
| 3 | Scope, lambda, map, filter | Transform and filter datasets |
| 4 | Recursion, generators, decorators | Implement 3 advanced examples |
| 5 | Classes, objects, self, __init__ | Build a Student class |
| 6 | Attributes and method types | Build an Experiment class |
| 7 | Encapsulation and properties | Validate model metrics |
| 8 | Inheritance and super() | Build parent and child model classes |
| 9 | Polymorphism and abstraction | Create different model interfaces |
| 10 | Composition and special methods | Build a small data-processing system |
| 11 | OOP interview questions | Answer 20 questions aloud |
| 12 | Function coding interview | Solve 5 problems without looking |
| 13 | OOP coding interview | Solve 5 class-design problems |
| 14 | AIML mini-project and revision | Explain your project from input to output |

## Final revision checklist

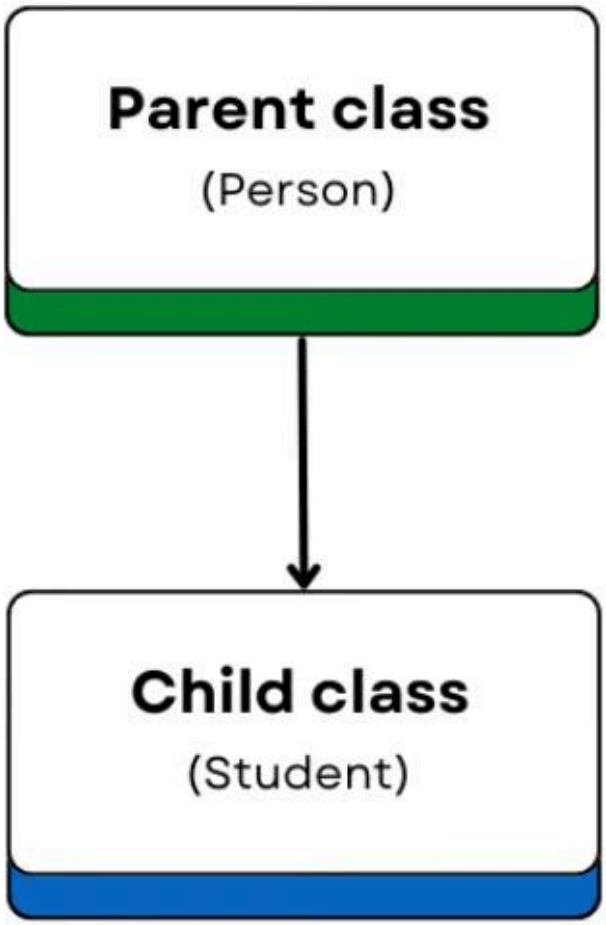
Before calling yourself interview-ready, check that you can do the following without notes:

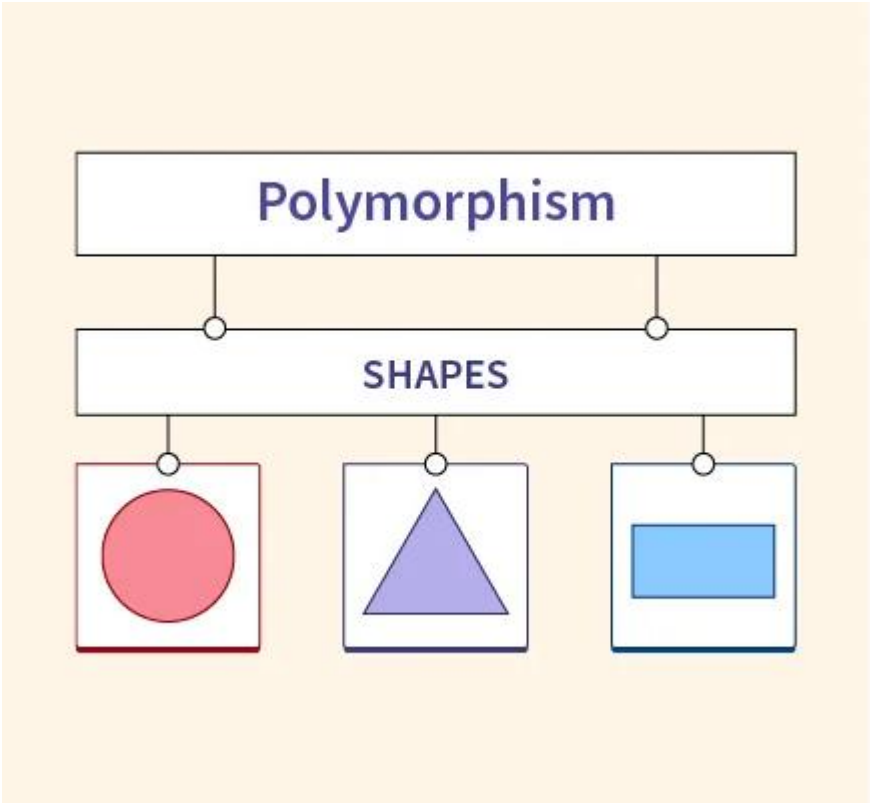
- Explain parameters, arguments, return, and print.
- Write functions using defaults, *args, and **kwargs.
- Explain scope, recursion, decorators, and generators.
- Create classes and objects using self and __init__().
- Demonstrate the four pillars of OOP with code.
- Explain inheritance, overriding, super(), and MRO.
- Choose between inheritance and composition.
- Build reusable preprocessing and evaluation components for an AIML project.
- Explain every line of your code and manually dry-run it.

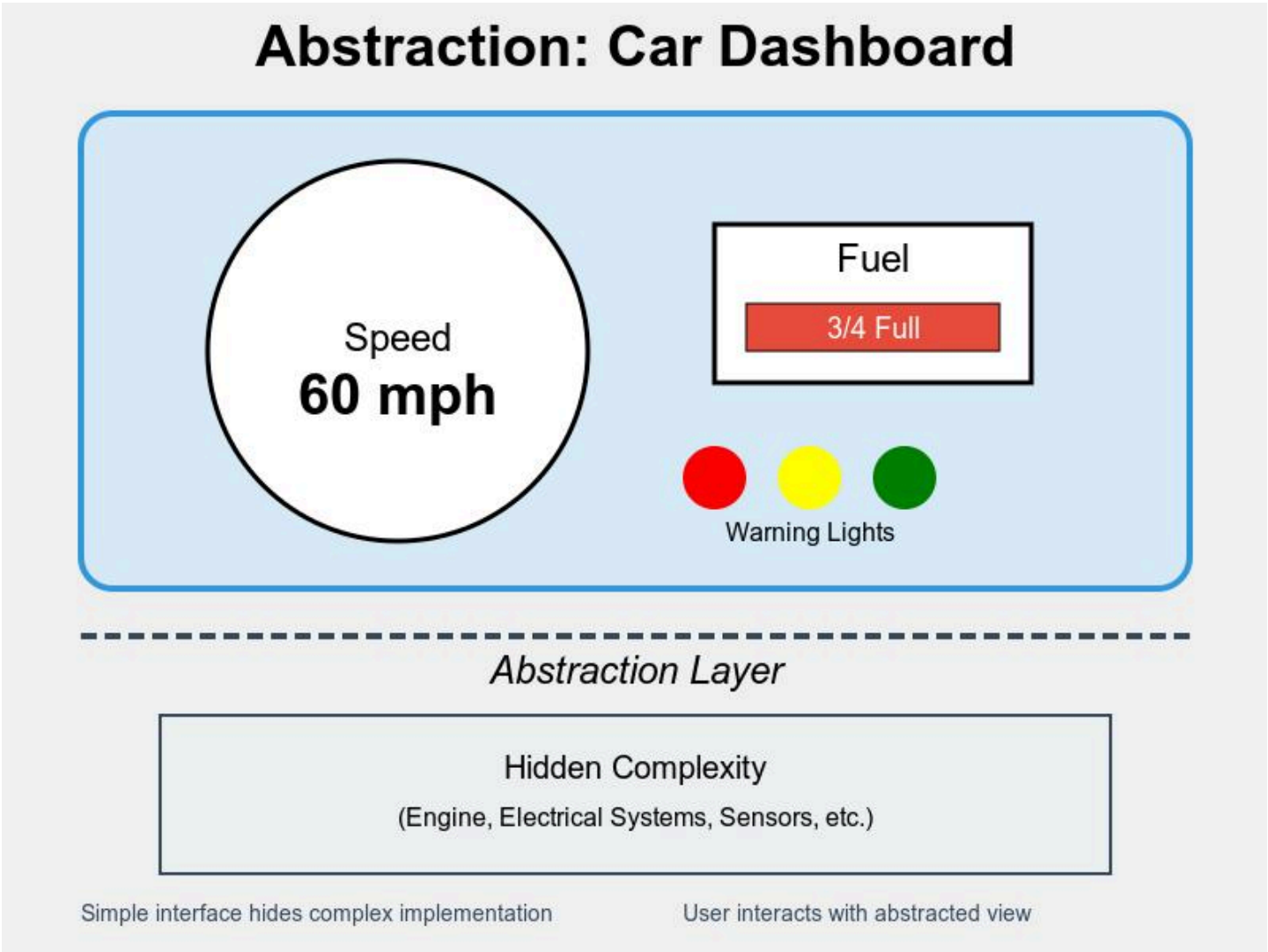
**My recommendation:** Master functions first, then learn OOP, then combine both in a small ML project. Functions help you write reusable logic; OOP helps you organize a larger system. Both are useful for Python-based AI/ML engineering.


Unsafe
Access
Outside
World
Direct
Modification
Data
Methods
Safe Access
Encapsulation

# SINGLE INHERITANCE




Polymorphism
SHAPES


Abstraction: Car Dashboard
Speed
60 mph
Fuel
3/4 Full
Warning Lights
Abstraction Layer
Hidden Complexity
(Engine, Electrical Systems, Sensors, etc.)
Simple interface hides complex implementation
User interacts with abstracted view

all oops tough word and concept explain through out the images and very easy word

all oops practical knowledge and concept

# Python OOP: Complete Practical Guide for AIML Engineers

Learn Object-Oriented Programming (OOP) from **zero to advanced level** in very easy English, with real-life examples, Python code, dry runs, diagrams, interview questions, and practical AI/ML applications.

The most important idea to remember is:

**OOP helps you organize code so that related data and actions stay together.**

For example, an ML experiment has a model name, parameters, training behavior, and prediction behavior. OOP helps organize these things into a class and its objects.

## 1. Understand OOP with a real-life example

Think about a car factory.

- **Class:** The design or blueprint for making cars.
- **Object:** An actual car made from that design.
- **Attributes:** Information about the car, such as color and speed.
- **Methods:** Actions the car can perform, such as starting and stopping.

The same idea works in Python.

```
class Car:
    def __init__(self, color, speed):
        self.color = color
        self.speed = speed

    def drive(self):
        print(f"The {self.color} car drives at {self.speed} km/h.")

car1 = Car("Red", 60)
car2 = Car("Blue", 80)

car1.drive()
car2.drive()
```

Output:

```
The Red car drives at 60 km/h.
The Blue car drives at 80 km/h.
```

Dry run:

| Code | Meaning |
|---|---|
| `class Car:` | Create a blueprint |
| `Car("Red", 60)` | Create the first car object |
| `self.color = color` | Store that car's color |
| `car1.drive()` | Run the first car's method |
| `car2.drive()` | Run the second car's method |

Each object has its own data, but both use the same class design.

# 2. Complete OOP chapter roadmap

# 3. Chapter 1 — Class, object, attributes and methods

## Easy definitions

| Term | Meaning in simple words |
|---|---|
| Class | A blueprint for creating objects |
| Object | A particular thing created from a class |
| Attribute | Data stored in an object or class |
| Method | A function written inside a class |
| Instance | Another word for an object created from a class |
| State | The current values stored in an object |

| Term | Meaning in simple words |
|---|---|
| Behavior | What an object can do |

### Practical example: Student management

```
class Student:
    def __init__(self, name, marks):
        self.name = name
        self.marks = marks

    def is_passed(self):
        return self.marks >= 40

    def display(self):
        print(f"Name: {self.name}")
        print(f"Marks: {self.marks}")
        print(f"Passed: {self.is_passed()}")

student1 = Student("Aman", 75)
student2 = Student("Riya", 32)

student1.display()
student2.display()
```

Output:

```
Name: Aman
Marks: 75
Passed: True
Name: Riya
Marks: 32
Passed: False
```

**Practical knowledge:** Use a class when several pieces of data and related operations belong together. A single simple calculation may be better as an ordinary function.

## 4. Chapter 2 — `__init__()`, `self` and object creation

### What is `__init__()`?

It is the initialization method that Python calls after creating a new object. We commonly use it to give the object its starting data.

## What is self?

self refers to the current object. It lets each object store and access its own attributes.

```
class MLExperiment:
    def __init__(self, model_name, learning_rate):
        self.model_name = model_name
        self.learning_rate = learning_rate

    def show_config(self):
        print(self.model_name, self.learning_rate)

exp1 = MLExperiment("NeuralNetwork", 0.01)
exp2 = MLExperiment("DecisionTree", 0.1)

exp1.show_config()
exp2.show_config()
```

Output:

```
NeuralNetwork 0.01
DecisionTree 0.1
```

## What happens internally?

When Python runs:

```
exp1 = MLExperiment("NeuralNetwork", 0.01)
```

Conceptually:

1. Python creates a new MLExperiment object.
2. Python calls __init__() with that object and the provided values.
3. self.model_name stores "NeuralNetwork".
4. self.learning_rate stores 0.01.
5. The new object is assigned to exp1.

Important distinction: __init__() initializes an object; __new__() is the method responsible for creating the object itself.

# 5. Chapter 3 — Instance, class and static methods

## 5.1 Instance method

Works with one particular object through `self`.

## 5.2 Class method

Uses `@classmethod` and receives the class as `cls`.

## 5.3 Static method

Uses `@staticmethod`. It does not automatically receive `self` or `cls`.

```
class MLExperiment:
    framework = "Python"

    def __init__(self, model_name):
        self.model_name = model_name

    def describe(self):
        return f"{self.model_name} uses {self.framework}"

    @classmethod
    def set_framework(cls, name):
        cls.framework = name

    @staticmethod
    def valid_learning_rate(rate):
        return 0 < rate <= 1

experiment = MLExperiment("RandomForest")

print(experiment.describe())
print(MLExperiment.valid_learning_rate(0.01))

MLExperiment.set_framework("Python ML")
print(experiment.describe())
```

Output:

```
RandomForest uses Python
True
RandomForest uses Python ML
```

**Interview tip:** Instance methods work with object data, class methods work with class-level data, and static methods are related utilities that need neither automatically.

# 6. Chapter 4 — Instance variables vs class variables

```
class Model:
    framework = "scikit-learn"  # Class variable

    def __init__(self, name):
        self.name = name        # Instance variable

m1 = Model("DecisionTree")
m2 = Model("RandomForest")

print(m1.name)
print(m2.name)
print(m1.framework)
print(m2.framework)
```

Output:

```
DecisionTree
RandomForest
scikit-learn
scikit-learn
```

- name belongs to each individual object.
- framework is defined at class level and can be shared by instances.

**Common mistake:** Do not use a shared mutable class attribute for data that each object should own independently.

```
class Experiment:
    def __init__(self):
        self.results =   # Each object gets its own list
```

# 7. Chapter 5 — The four pillars of OOP

These four concepts are the foundation of OOP interviews.

## Pillar 1: Encapsulation

**Easy meaning:** Keep data and the methods that work with it together, and control how data is changed.

Real-world example: A bank account should not allow someone to withdraw an arbitrary negative amount.

```
class BankAccount:
    def __init__(self, balance=0):
        if balance < 0:
            raise ValueError("Initial balance cannot be negative")
        self._balance = balance

    @property
    def balance(self):
        return self._balance

    def deposit(self, amount):
        if amount <= 0:
            raise ValueError("Deposit must be positive")
        self._balance += amount

account = BankAccount(1000)
account.deposit(500)
print(account.balance)
```

Output: 1500

Important terms:

- _balance: A convention that the attribute is intended for internal use.
- @property: Lets you read a method through attribute-style access.
- Validation: Checking that incoming values are acceptable.

Python does not enforce full privacy with a single underscore. Encapsulation is mainly about providing a safe and clear interface.

## Pillar 2: Inheritance

**Easy meaning:** One class can reuse and extend another class's behavior.

```
class MLModel:
    def __init__(self, name):
        self.name = name

    def describe(self):
        print("Model:", self.name)

class Classifier(MLModel):
    def predict(self, features):
        return "Class A"

model = Classifier("ImageClassifier")
model.describe()
print(model.predict([0.2, 0.7]))
```

Output:

```
Model: ImageClassifier
Class A
```

Classifier inherits the initialization and description behavior from MLModel.

Common inheritance types:

- **Single:** One child inherits from one parent.
- **Multilevel:** A child inherits from a class that already inherits from another class.
- **Multiple:** A class inherits from more than one parent.
- **Hierarchical:** Multiple child classes inherit from one parent.
- **Hybrid:** A combination of these patterns.

## Pillar 3: Polymorphism

**Easy meaning:** The same operation can work differently for different objects.

```
class Classifier:
    def predict(self, features):
        return "Cat"

class Regressor:
    def predict(self, features):
        return 23.5

models = [Classifier(), Regressor()]

for model in models:
    print(model.predict([1, 2, 3]))
```

Output:

```
Cat
23.5
```

Both objects have a `predict()` method, but their results are different.

Practical AIML application: Your application can call `predict()` on different model types without needing to know every detail of their implementations.

### Method overloading vs overriding

- **Overriding:** A child class replaces or extends an inherited method.
- **Overloading:** Traditionally, defining multiple methods with the same name but different parameter signatures. Python does not support Java-style signature-based method overloading directly. You can often achieve similar behavior with default arguments, `*args`, or `functools.singledispatch`.

## Pillar 4: Abstraction

**Easy meaning:** Show the user what an object can do without requiring them to understand every internal detail.

```
from abc import ABC, abstractmethod

class Predictor(ABC):
    @abstractmethod
    def predict(self, features):
        pass

class SimplePredictor(Predictor):
    def predict(self, features):
        return sum(features) / len(features)

model = SimplePredictor()
print(model.predict([10, 20, 30]))
```

Output: 20.0

The abstract class defines a requirement: a concrete predictor must provide predict().

The actual calculation is only a demonstration, not a trained ML model.

**Remember the difference:**

| Concept | Easy question it answers |
|---|---|
| Encapsulation | How do I control access to data? |
| Inheritance | How can I reuse and extend a class? |
| Polymorphism | How can the same operation behave differently? |
| Abstraction | How do I expose essential behavior and hide implementation details? |

# 8. Chapter 6 — Method overriding and super()

## What is method overriding?

When a child class provides its own version of a method that comes from its parent, that is method overriding.

```
class Model:
    def predict(self):
        return "Basic prediction"

class NeuralNetwork(Model):
    def predict(self):
        return "Neural network prediction"

model = NeuralNetwork()
print(model.predict())
```

Output: Neural network prediction

## What is super()?

super() lets you access the next method in the inheritance lookup order, commonly to reuse a parent's initialization or behavior.

```
class Model:
    def __init__(self, name):
        self.name = name

    def describe(self):
        return f"Model: {self.name}"

class NeuralNetwork(Model):
    def __init__(self, name, layers):
        super().__init__(name)
        self.layers = layers

    def describe(self):
        return f"{super().describe()}, layers={self.layers}"

model = NeuralNetwork("ImageModel", 5)
print(model.describe())
```

Output:

```
Model: ImageModel, layers=5
```

**Why use super()?** It avoids repeating parent initialization code and supports cooperative inheritance patterns.

# 9. Chapter 7 — MRO (Method Resolution Order)

MRO means **the order Python follows to find a method or attribute** when a class inherits from other classes.

```
class A:
    def show(self):
        print("A")

class B(A):
    def show(self):
        print("B")

class C(A):
    def show(self):
        print("C")

class D(B, C):
    pass

obj = D()
obj.show()

print([cls.__name__ for cls in D.mro()])
```

Output:

```
B
['D', 'B', 'C', 'A', 'object']
```

Python finds `show()` in `B` first, so it prints `B`.

This example also demonstrates multiple inheritance: `D` inherits from both `B` and `C`.

**Practical tip:** Use multiple inheritance carefully. When several parents define similar methods, understand the MRO and use `super()` consistently when cooperative calls are intended.

# 10. Chapter 8 — Composition vs inheritance

These are two ways to build relationships between classes.

- Inheritance: A classifier **is a kind of** model.

- Composition: A prediction service **uses a** preprocessor.

```
class Preprocessor:
    def transform(self, values):
        return [value / 100 for value in values]

class PredictionService:
    def __init__(self, preprocessor):
        self.preprocessor = preprocessor

    def prepare(self, values):
        return self.preprocessor.transform(values)

preprocessor = Preprocessor()
service = PredictionService(preprocessor)

print(service.prepare([20, 50, 80]))
```

Output:

```
[0.2, 0.5, 0.8]
```

The prediction service uses a preprocessor object. It does not inherit from Preprocessor.

| Choose inheritance when... | Choose composition when... |
| --- | --- |
| One type really is a subtype of another | One object uses another component |
| You need a shared interface or behavior | You want to replace components easily |
| The parent-child relationship is natural | You want fewer dependencies between classes |

In practical ML applications, composition is often useful for assembling data cleaning, feature processing, prediction, and evaluation components.

# 11. Chapter 9 — Special methods (dunder methods)

Dunder means **double underscore**. These are special methods Python uses for built-in operations.

Example:

```
class Dataset:
    def __init__(self, samples):
        self.samples = samples

    def __len__(self):
        return len(self.samples)

    def __str__(self):
        return f"Dataset containing {len(self)} samples"

data = Dataset([10, 20, 30, 40])

print(len(data))
print(data)
```

Output:

```
4
Dataset containing 4 samples
```

Here, `len(data)` works because we defined `__len__()`, and `print(data)` uses `__str__()`.

## 12. Chapter 10 — Properties, getters and setters

A **property** allows you to access a method as if it were an attribute.

This is useful when a value needs validation or calculation.

```
class ModelMetrics:
    def __init__(self, accuracy):
        self.accuracy = accuracy

    @property
    def accuracy(self):
        return self._accuracy

    @accuracy.setter
    def accuracy(self, value):
        if not 0 <= value <= 1:
            raise ValueError("Accuracy must be between 0 and 1")
        self._accuracy = value

metrics = ModelMetrics(0.95)
print(metrics.accuracy)

metrics.accuracy = 0.97
print(metrics.accuracy)
```

Output:

```
0.95
0.97
```

If you try to assign `metrics.accuracy = 1.5`, Python raises a `ValueError`.

**Easy memory trick:** A getter reads a value; a setter controls how a value is changed.

# 13. Chapter 11 — Dataclasses

A dataclass is a convenient way to create classes mainly used to store data. Python can automatically generate methods such as `__init__()` and `__repr__()`.

```
from dataclasses import dataclass

@dataclass
class ExperimentConfig:
    model_name: str
    learning_rate: float
    epochs: int

config = ExperimentConfig("NeuralNetwork", 0.001, 20)

print(config)
print(config.learning_rate)
```

Output:

```
ExperimentConfig(model_name='NeuralNetwork', learning_rate=0.001, epochs=20)
0.001
```

AIML use case: Storing the model name, learning rate, number of epochs, batch size, and other experiment settings in one organized object.

## 14. Chapter 12 — Exceptions and error handling in OOP

An exception is an error or unusual event that occurs while a program runs. Handle expected failures clearly instead of letting invalid inputs silently produce bad results.

```
class DataProcessor:
    def transform(self, values):
        if not values:
            raise ValueError("Dataset cannot be empty")

        return [value / 100 for value in values]

processor = DataProcessor()

try:
    print(processor.transform([20, 40, 60]))
    print(processor.transform())
except ValueError as error:
    print("Error:", error)
```

Output:

```
[0.2, 0.4, 0.6]
Error: Dataset cannot be empty
```

Key words:

- try: Code that might fail.
- except: Handles a matching exception.
- raise: Intentionally raises an exception.
- finally: Code that runs whether an exception occurs or not.

For AIML projects, validate input shape, missing values, data types, and expected feature names before making predictions.

---

# 15. Practical AIML project — build an object-oriented ML pipeline

Now let's use OOP concepts together in a project structure that resembles real ML work.

This example uses scikit-learn. It trains a real regression model to predict a numeric value from a single feature. The data is **synthetic demonstration data**, not a real-world dataset.

If you don't have the package, install it using:

```
pip install scikit-learn
```

## Complete code

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_absolute_error
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler

# Function: check the input data
def validate_features(X):
    if len(X) == 0:
        raise ValueError("Training data cannot be empty")
    if any(len(row) != 1 for row in X):
        raise ValueError("Each row must contain one feature")
    return X

# Class: organize the training and prediction operations
class HousePriceModel:
    def __init__(self):
        self.pipeline = Pipeline([
            ("scaler", StandardScaler()),
            ("model", LinearRegression())
        ])

    def train(self, X, y):
        X = validate_features(X)

        if len(X) != len(y):
            raise ValueError("X and y must have equal lengths")

        self.pipeline.fit(X, y)
        return self

    def predict(self, X):
        X = validate_features(X)
        return self.pipeline.predict(X)

# Demonstration dataset: house size in square feet
X_train = [[500], [700], [900], [1100], [1300]]
y_train = [100, 140, 180, 220, 260]  # Example price units

X_test = [[800], [1000]]
y_test = [160, 200]

# Create an object
model = HousePriceModel()

# Train the model
model.train(X_train, y_train)

# Predict on held-out demonstration samples
```

```
predictions = model.predict(X_test)

# Evaluate prediction errors
mae = mean_absolute_error(y_test, predictions)

print("Predictions:", predictions)
print("Actual values:", y_test)
print("MAE:", mae)
```

Expected result (approximately):

```
Predictions: [160. 200.]
Actual values: [160, 200]
MAE: 0.0
```

The synthetic data follows an exact linear relationship, so a linear regression model can learn it perfectly. Real-world data usually contains noise and more complex relationships.

## Identify the OOP concepts in the project

| Code | OOP or Python concept |
|---|---|
| `class HousePriceModel` | Class |
| `model = HousePriceModel()` | Object creation |
| `self.pipeline` | Instance attribute |
| `__init__()` | Initialization |
| `train()` | Instance method |
| `predict()` | Instance method |
| `validate_features()` | Reusable function |
| `Pipeline(...)` | Composition of ML processing steps |
| `self.pipeline.fit(...)` | Calling a model's training behavior |

**Important practical lesson:** `train()` and `predict()` are separate responsibilities. The same model object retains its trained state so you can make later predictions without retraining every time.

In a production system, you would also check target values, missing data, feature consistency, train/test leakage, and whether the chosen metric matches the business goal.

# 16. Advanced OOP terms you should know

These are useful for more advanced interviews and larger Python projects.

| Term | Meaning in very easy words | Practical use |
|---|---|---|
| object | The common base class for ordinary Python classes | General object behavior |
| isinstance() | Checks whether an object is an instance of a class or its subclass | Input validation |
| issubclass() | Checks whether a class inherits from another class | Checking class relationships |
| hasattr() | Checks whether an object has a named attribute | Checking optional features |
| getattr() | Reads an attribute by name | Flexible configuration |
| setattr() | Sets an attribute by name | Dynamic configuration |
| __dict__ | Often holds an object's or class's attribute dictionary | Debugging and inspection |
| MRO | The order Python follows to look up methods | Multiple inheritance |
| Duck typing | Use an object based on what it can do | Interchangeable model components |
| Dependency injection | Give a class the components it needs from outside | Testing and swapping ML models |
| SOLID | Five principles for designing maintainable OOP systems | Larger software projects |
| Unit testing | Testing a small piece of code independently | Checking prediction and validation logic |

## Dependency injection: a practical example

Instead of making a prediction service create its own model, pass the model into it.

```
class PredictionService:
    def __init__(self, model):
        self.model = model

    def predict(self, features):
        return self.model.predict(features)

class DemoModel:
    def predict(self, features):
        return sum(features)

service = PredictionService(DemoModel())
print(service.predict([10, 20, 30]))
```

Output: 60

This design makes it easier to replace DemoModel with a real ML model or a test model.

### SOLID principles — simple meanings

- **S — Single Responsibility:** A class should have one clear main responsibility.
- **O — Open/Closed:** Make it possible to add new behavior without constantly modifying working code.
- **L — Liskov Substitution:** A child class should be usable wherever its parent type is expected without breaking expected behavior.
- **I — Interface Segregation:** Prefer small, focused interfaces over one huge interface.
- **D — Dependency Inversion:** High-level logic should depend on abstractions rather than tightly depending on specific implementations.

You do not need to memorize all the formal definitions first. Understand the problem each principle helps solve.

## 17. OOP interview practice

Try answering these questions aloud before opening the answers.

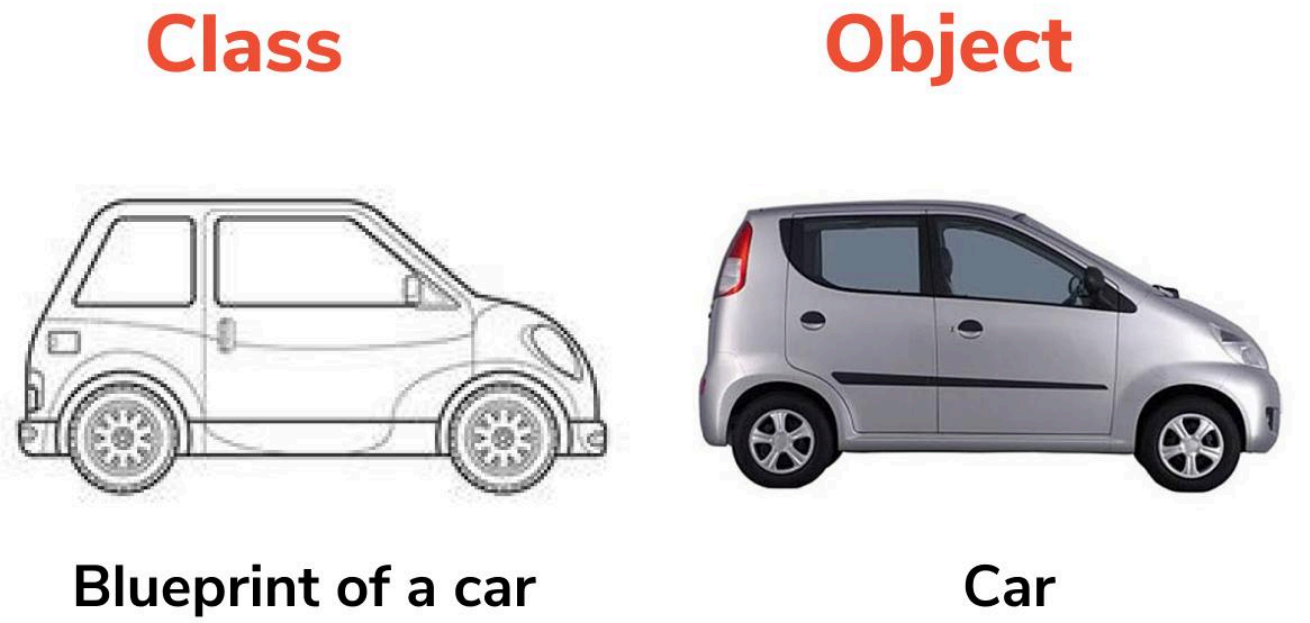
## 18. Your practical study plan

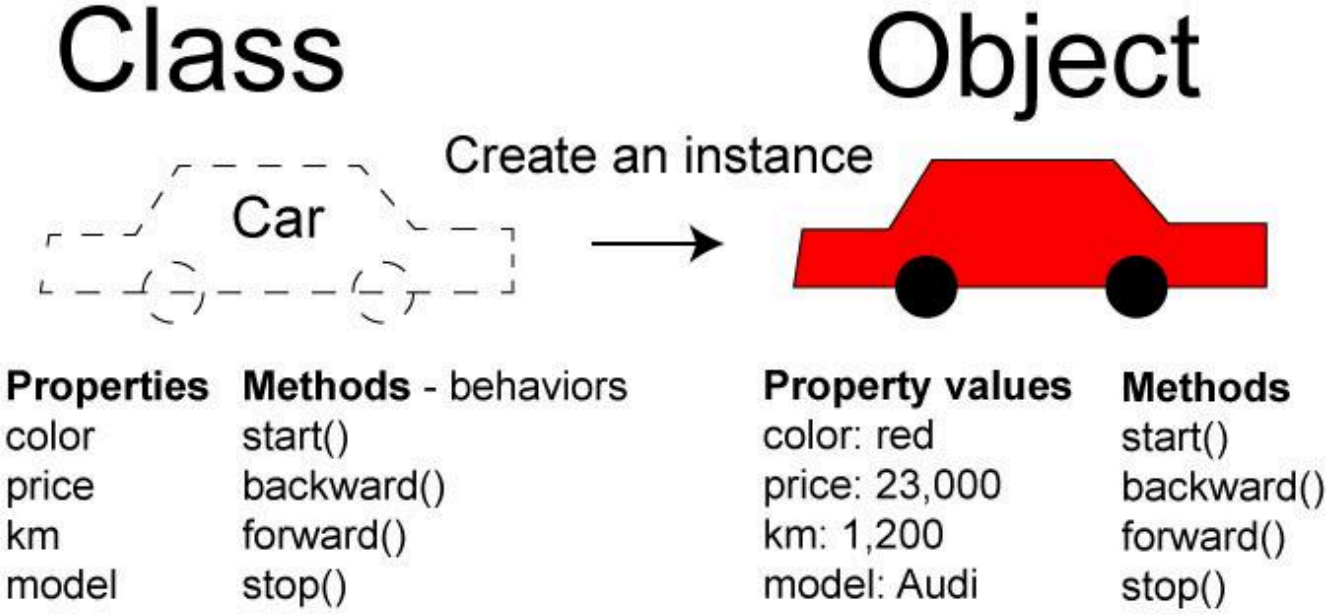
### OOP mastery checklist

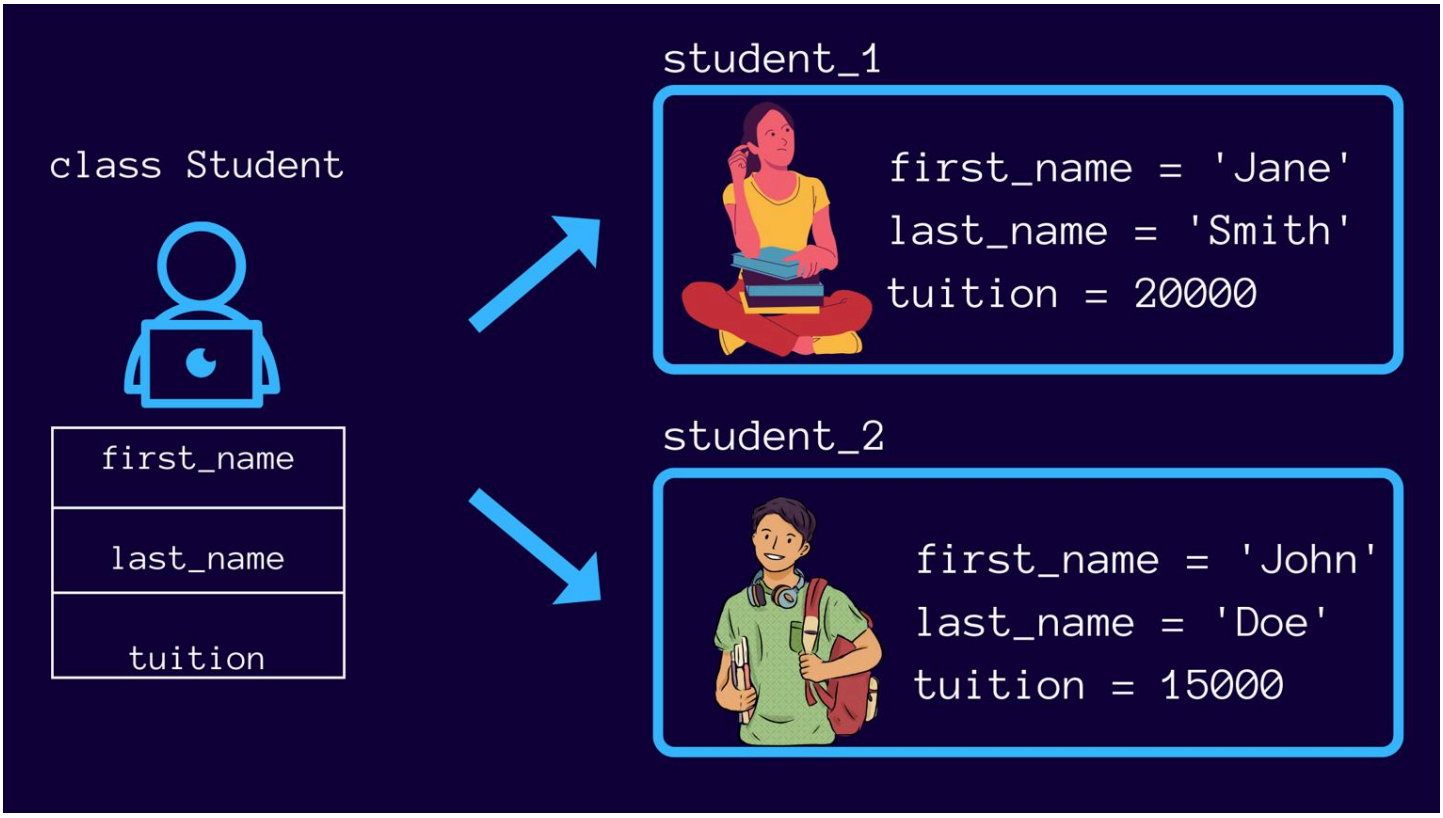
Your progress

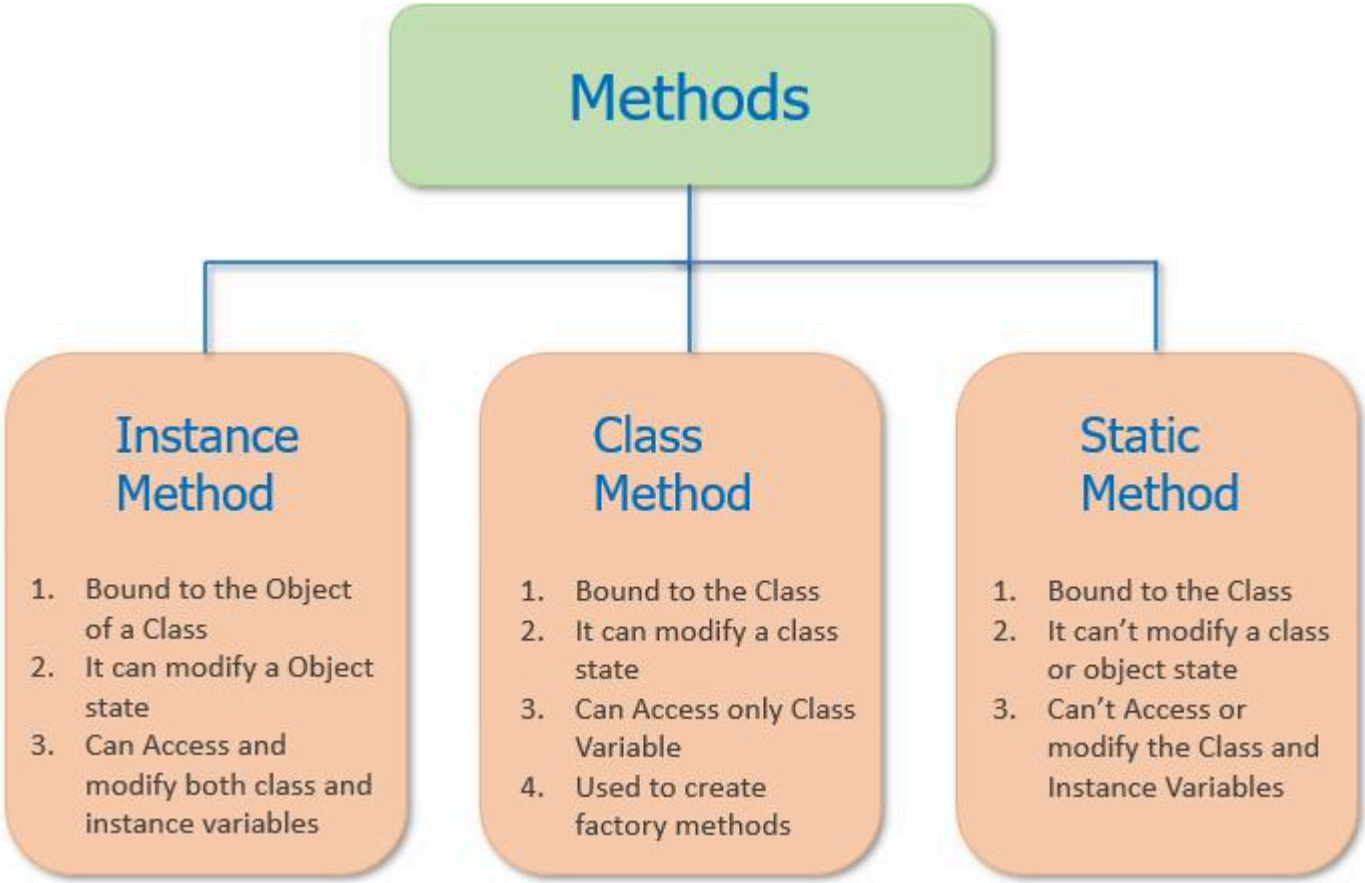
/

**My recommendation:** Study two or three concepts at a time. For every concept, write a small program, run it, change the values, predict the output, and explain why it works. Once the basics feel comfortable, build the ML pipeline above and explain how each class and method contributes to the workflow.




class Student
first_name
last_name
tuition
student_1
first_name = 'Jane'
last_name = 'Smith'
tuition = 20000
student_2
first_name = 'John'
last_name = 'Doe'
tuition = 15000


Methods
Instance Method
1. Bound to the Object of a Class
2. It can modify a Object state
3. Can Access and modify both class and instance variables
Class Method
1. Bound to the Class
2. It can modify a class state
3. Can Access only Class Variable
4. Used to create factory methods
Static Method
1. Bound to the Class
2. It can't modify a class or object state
3. Can't Access or modify the Class and Instance Variables

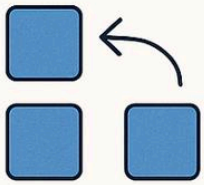
# PRINCIPLES OF OBJECT-ORIENTED PROGRAMMING

## ENCAPSULATION



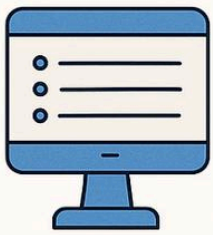
Bundling of data and methods that operate on the data within one unit

## POLY-MORPHISM



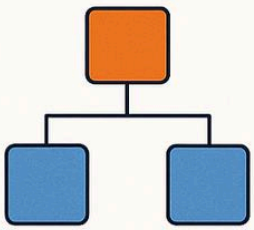
Ability of different objects to be treated as instances of the same class

## ABSTRACTION

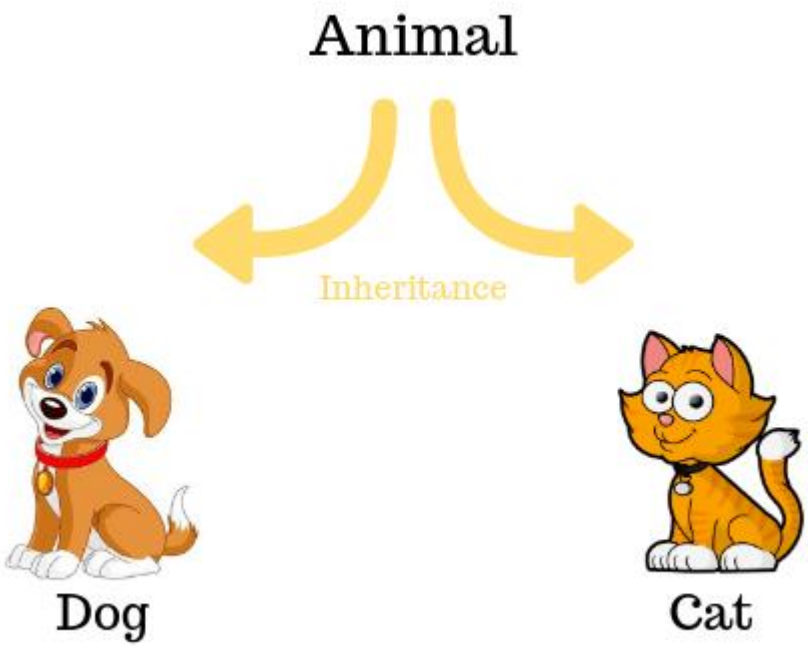


Hiding complex implementation details and showing only essential features

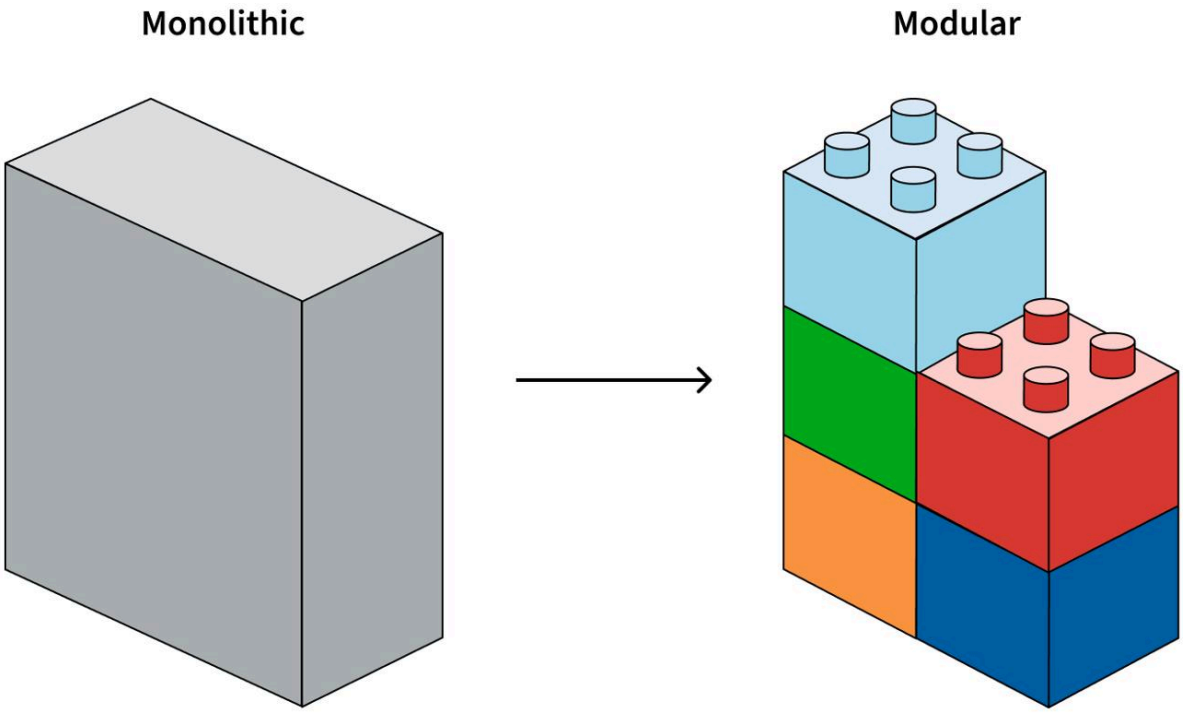
## INHERITANCE

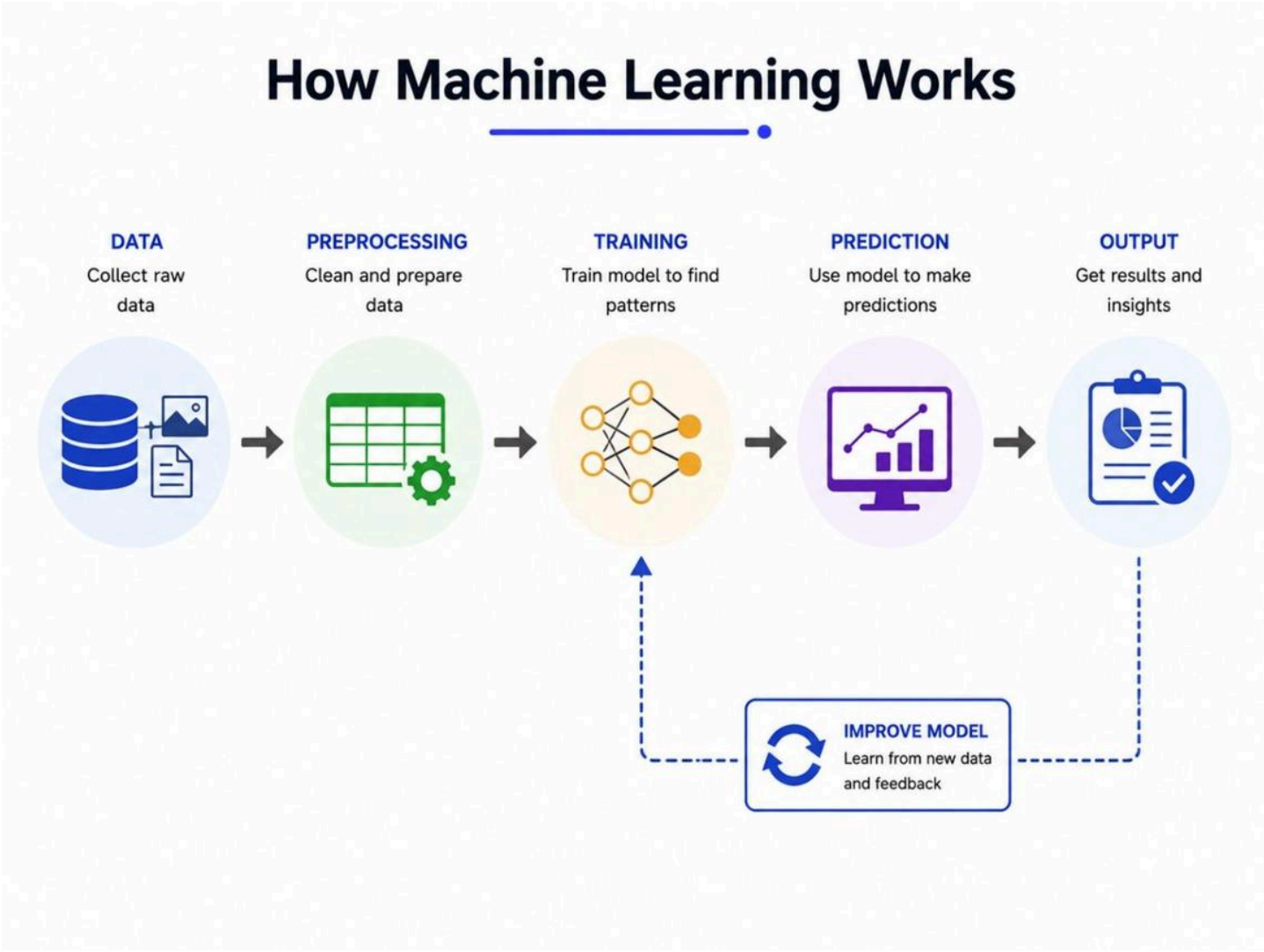


Mechanism of basing an object or class upon another class






Monolithic
Modular


How Machine Learning Works
DATA
Collect raw data
PREPROCESSING
Clean and prepare data
TRAINING
Train model to find patterns
PREDICTION
Use model to make predictions
OUTPUT
Get results and insights
IMPROVE MODEL
Learn from new data and feedback